

CG作业6报告

李惟乐 PB23050929

内容

- [算法原理](#)
- [结果](#)

算法原理

1. Blinn-Phong模型

模型组成

Blinn-Phong模型将光照分为三个部分叠加：

$$I = I_{\text{ambient}} + I_{\text{diffuse}} + I_{\text{specular}}$$

环境光 (Ambient)

环境光模拟间接光照，与表面法线和光源方向无关：

$$I_{\text{ambient}} = k_a \cdot I_a$$

- k_a : 环境光反射系数
- I_a : 环境光强度

由以下代码实现：

```
// ambient part
vec3 albedo = texture(diffuseColorSampler, uv).rgb;
vec3 ambient = k_a * albedo;
```

漫反射 (Diffuse)

漫反射与表面法线 $\mathbf{n}$ 和光源方向 $\mathbf{l}$ 的夹角相关：

$$I_{\text{diffuse}} = k_d \cdot I_l \cdot (\mathbf{n} \cdot \mathbf{l})$$

- k_d : 漫反射系数
- I_l : 光源强度
- $\mathbf{n} \cdot \mathbf{l}$: 法线与光源方向的点积，结果映射到 $[0, 1]$

由以下代码实现：

```
// diffuse part
vec3 norm = normalize(normal);
vec3 lightDir = normalize(light.position - pos);
float diff = max(dot(norm, lightDir), 0.0);
vec3 diffuse = k_d * light.color * (diff * albedo);
```

高光反射 (Specular)

Blinn-Phong采用半角向量 $\mathbf{h}$ 替代Phong模型中的反射向量：

$$I_{\text{specular}} = k_s \cdot I_l \cdot (\mathbf{n} \cdot \mathbf{h})^\alpha$$

- k_s : 高光反射系数
- α : 光泽度 (shininess)，控制高光锐利度
- 半角向量 $\mathbf{h}$: 视线方向 $\mathbf{v}$ 和光源方向 $\mathbf{l}$ 的归一化中间向量：

$$\mathbf{h} = \frac{\mathbf{v} + \mathbf{l}}{\|\mathbf{v} + \mathbf{l}\|}$$

由以下代码实现：

```
// specular part
float alpha = (1.0 - roughness) * 150;
vec3 viewDir = normalize(camPos - pos);
vec3 halfDir = normalize(lightDir + viewDir);
float spec = 0.0;
if (abs(diff) > 1e-10)
{
    spec = pow(max(dot(norm, halfDir), 0.0), alpha);
}
vec3 specular = k_s * spec * light.color;
```

参数选取

在本次作业中，选取：

- $k_s = \text{metal} * 0.8$
- $k_d = 1 - k_s$
- $\alpha = (1 - \text{roughness}) * \text{const1}$
- $k_a = \text{const2}$

2. Shadow Mapping

Shadow Mapping核心分为两个阶段：

- **阴影图生成阶段**：将虚拟摄像机置于光源位置，记录场景中各点到光源的深度值
- **阴影渲染阶段**：从观察视角渲染场景时，将像素点转换到光源空间进行深度比较，判断是否被遮挡

在本次作业中，通过函数 `GetShadow` 来返回当前点是否在阴影中。

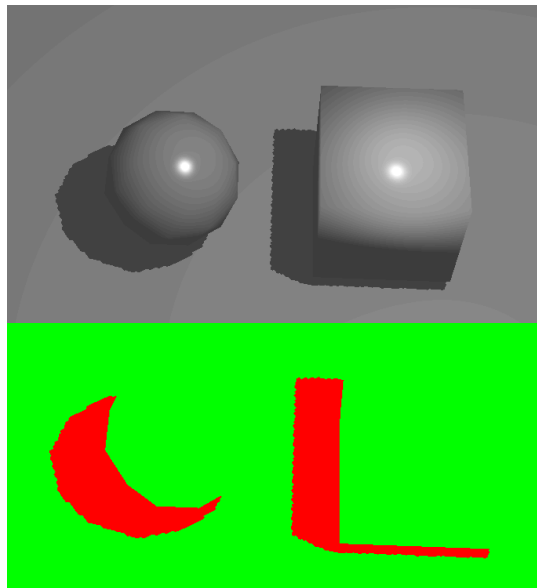
```
float GetShadow(vec4 lightSpacePos, int lightIndex, float diff)
{
    float bias = max(0.05 * (1.0 - diff), 0.005);
    vec3 projCoords = lightSpacePos.xyz / lightSpacePos.w;
    float currentDepth = projCoords.z;
    projCoords = projCoords * 0.5 + 0.5;
    float closestDepth = texture(shadow_maps, vec3(projCoords.xy, lightIndex)).r;
    return currentDepth - bias > closestDepth ? 1.0 : 0.0;
}
```

其中 `ambient` 部分不受影响，其他两部分受影响

```
vec3 lighting = (diffuse + specular) * (1.0 - 0.90 * shadow);
```

结果

从上至下分别为效果图和阴影图，阴影图中红色表示阴影，绿色表示非阴影





- 注：在第一个示例中平面法向量默认是向下的，其与光源的点乘结果小于零，故会取零导致无阴影。因此在第一个示例中将平面移动到物品上方，旋转相机即可得到正确的结果。